

ADVANCED AUTOMATION ENGINEERING PROGRAM

Navigation, Frames & Accessibility

Multi-page flows · tabs & popups · iframes · dialogs · Axe a11y

SPRINT 1

DAY 4 of 42

Jai Singh

Week 1 — UI Automation (Playwright) · Full-day session (3h teach + 5h hands-on)

Planner ref: Week 1 · Day 4 — Navigation, Frames & Accessibility

01 How Today Runs

SESSION MAP

Day 3 made a single page stable. Real flows span many pages, open new tabs, embed third-party frames, and pop up dialogs — and they must be usable by everyone. Today we automate those flows and check accessibility.

09:30	 Concepts	Navigation, lifecycle, tabs/popups, context model
11:00	 Frames, UI & a11y, live	iframes, dropdowns, dialogs, Axe — in the IDE
12:00	 Guided exercises	Short 'you try' tasks with checkpoints
13:30	 Lab (staged)	Catalog → product → cart, frame, dialog, Axe scan
16:30	 Debrief & recap	Flow pitfalls, evaluation, Day-5 preview

Real-world UI flows: moving between pages, handling new tabs and frames, driving dropdowns and dialogs, and verifying accessibility.



Navigation

Move between pages & URLs



Tabs & popups

New windows and pop-ups



Frames / iframes

Act inside embedded docs



Dropdowns & dialogs

selectOption, alerts, confirms



Page lifecycle

load, domcontentloaded events



Accessibility

Axe-core checks on journeys

Aligned to Planner Wk1/Day4 “Navigation, Frames & Accessibility”. ToC: navigation, multi-tab, frames, dropdowns/alerts/popups, lifecycle, Axe a11y.

Deferred: debugging/traces/parallel→D5.

By the end of today you will be able to:

- ✓ Navigate between pages and assert URL / title transitions reliably.
- ✓ Handle new tabs and pop-ups via the BrowserContext, not brittle window indices.
- ✓ Act on elements inside an iframe using frameLocator.
- ✓ Drive dropdowns and respond to native dialogs (alert / confirm / prompt).
- ✓ Use page lifecycle events and waitForURL to sync on navigation.
- ✓ Run an Axe-core accessibility scan on a key journey and read the violations.

Navigation is more than `goto()`. Playwright tracks the page lifecycle and lets you wait for the right moment — usually you just assert and let it auto-wait.

```
await page.goto('/catalog');
await page.getByRole('link', { name: 'Running Shoes' }).click();
// assert the transition – waitForURL syncs on the new page
await page.waitForURL(/\\/product\\/);
await expect(page).toHaveTitle(/Running Shoes/);
```

Lifecycle states: `commit` (response started) · `domcontentloaded` (HTML parsed) · `load` (all resources). Prefer asserting UI / URL over waiting on raw events.

When a click opens a new tab or pop-up, wait for the context's 'page' event — set up the wait BEFORE the click that triggers it (just like a network response).

```
// the click opens a new tab
const [newPage] = await Promise.all([
  page.context().waitForEvent('page'),
  page.getByRole('link', { name: 'Size guide' }).click(),
]);
await newPage.waitForLoadState();
await expect(newPage).toHaveURL(/size-guide/);
```

Each tab is its own Page object inside the same BrowserContext — no window handles, no indices.

A context is an isolated browser session. Pages (tabs) live inside it and share cookies and storage — this is how Playwright handles multi-tab cleanly.



BrowserContext

One isolated session (like a fresh, private profile). Owns cookies, storage and permissions.



Pages inside it

Each tab / pop-up is a Page in the context. Reach them via the 'page' event or `context.pages()`.



From Selenium: no more `driver.getWindowHandles()` and `switchTo().window(handle)` — you hold a reference to each Page directly.

Tests usually run in their own context for isolation — one test's tabs never leak into another.

Selenium's `switchTo()` family maps to direct Playwright references and locators — no manual context switching.

SELENIUM (JAVA)		PLAYWRIGHT
<code>getWindowHandles() / switchTo().window</code>	→	<code>context 'page' event → Page ref</code>
<code>switchTo().frame("id")</code>	→	<code>page.frameLocator('#id')</code>
<code>switchTo().defaultContent()</code>	→	<code>(not needed – frameLocator is scoped)</code>
<code>switchTo().alert().accept()</code>	→	<code>page.once('dialog', d => d.accept())</code>
<code>new Select(el).selectByVisibleText</code>	→	<code>locator.selectOption({ label })</code>
<code>driver.navigate().to(url)</code>	→	<code>page.goto(url)</code>
<code>driver.navigate().back()</code>	→	<code>page.goBack()</code>

Key shift: Selenium switches the driver's focus; Playwright keeps separate handles and scoped locators — less state to get wrong.

Validate each transition rather than assuming it happened. This is the heart of today's hands-on.

```
// catalog → product
await page.getByRole('link', { name: 'Running Shoes' }).click();
await expect(page.getByRole('heading', { name: 'Running Shoes' })).toBeVisible();
// product → cart
await page.getByRole('button', { name: 'Add to cart' }).click();
await expect(page.getByTestId('cart-count')).toHaveText('1');
// go back – still works
await page.goBack();
```

09 Frames & iframes

CONCEPT

An iframe is a page embedded inside the page (payment widgets, maps, third-party content). Use `frameLocator` to reach elements inside it.

```
// scope into the frame, then locate as usual
const pay = page.frameLocator('#payment-frame');
await pay.getByLabel('Card number').fill('4111 1111 1111 1111');
await pay.getByRole('button', { name: 'Pay' }).click();
```

 **Note:** a normal page locator will NOT find elements inside an iframe — you must go through `frameLocator` first.

frameLocator chains for nested frames, and it auto-waits like any locator — no switchTo / defaultContent dance.

```
// frame within a frame — just chain
await page.frameLocator('#outer')
    .frameLocator('#inner')
    .getByRole('button', { name: 'Confirm' }).click();
```



From Selenium: no switchTo().frame(...) then switchTo().defaultContent() — the chain is scoped and self-contained.

Native <select>

selectOption() by label / value / index

Custom (div-based)

click to open, then click the option by role/text

```
// native <select>
await page.getByLabel('Size').selectOption({ label: 'UK 9' });
// custom dropdown — open, then pick
await page.getByRole('button', { name: 'Colour' }).click();
await page.getByRole('option', { name: 'Teal' }).click();
```

Check the HTML: a real <select> uses selectOption; a styled div menu is just click-to-open then click-option.

Native dialogs block the page. Register a handler BEFORE the action that triggers them — Playwright auto-dismisses if you do not.

```
// register first; once() handles a single expected dialog
page.once('dialog', async dialog => {
  expect(dialog.message()).toContain('Remove this item?');
  await dialog.accept(); // or dialog.dismiss()
});
await page.getByRole('button', { name: 'Remove' }).click();
```

For prompts, pass text to accept: `dialog.accept('my answer')`.

13 Catalog → Product → Cart

HANDS-ON



IN THE IDE build the core navigation flow together — the spine of today's lab.

```
test('select a product and add it to the cart', async ({ page }) => {
  await page.goto('/catalog');
  await page.getByRole('link', { name: 'Running Shoes' }).click();
  await page.getByLabel('Size').selectOption({ label: 'UK 9' });
  await page.getByRole('button', { name: 'Add to cart' }).click();
  await expect(page.getByTestId('cart-count')).toHaveText('1');
});
```

Pattern: navigate → pick a variant → act → assert the transition at every step.

14 Run It & Read the Result

HANDS-ON



IN THE IDE run headed to watch the navigation, then open the HTML report. Trace debugging comes tomorrow.

```
npx playwright test nav.spec.ts --headed
```

Watch the flow navigate

```
npx playwright test nav.spec.ts
```

Run headless

```
npx playwright show-report
```

Open the HTML report

What to look for: each transition is asserted, so a broken step fails exactly where it breaks — not three steps later.

Accessibility (a11y) means the app is usable by people with disabilities — and it is often a legal requirement. Automated checks catch a meaningful share of issues early.



What a11y covers

Colour contrast, labels for inputs, alt text, keyboard access, ARIA roles and names.



Why automate it

Axe catches many common, automatically detectable issues early — fast, repeatable, CI-friendly. Manual and assistive-tech review are still needed for full coverage.

Tie-in: the role-based locators you have used since Day 2 depend on the same accessibility tree Axe inspects — good a11y makes tests easier to write.

16 Axe-core — Install & First Scan

HANDS-ON



IN THE IDE install the Axe Playwright package, then scan a page and read the violations.

```
# one-time install
npm install -D @axe-core/playwright

import AxeBuilder from '@axe-core/playwright';

const results = await new AxeBuilder({ page }).analyze();
expect(results.violations).toEqual([]);
```

A failing assertion lists each violation with its rule id, impact, and the offending nodes.

Scope a scan to the part of the page you care about, and tag the rule sets (e.g. WCAG 2 AA) you want to enforce.

```
const results = await new AxeBuilder({ page })
  .include('#checkout-form') // scope to a region
  .withTags(['wcag2a', 'wcag2aa']) // rule sets
  .analyze();

// inspect the first violation when one is found
console.log(results.violations[0]?.id, results.violations[0]?.impact);
```

Each violation has: id, impact (minor→critical), help URL, and the exact nodes — enough to file or fix.

GUIDED PRACTICE

Guided Exercises

Three short 'you try' tasks on navigation, frames, and accessibility. Attempt each in the IDE, then we review together.

~30 minutes · individual · ask for help anytime

Clicking 'Size guide' opens a new tab. Capture that tab and assert its URL — without using window indices.

Your task:

- Wait for the context 'page' event around the click.
- Assert the new tab's URL contains 'size-guide'.



IN THE IDE use Promise.all with context.waitForEvent('page').

```
// TODO: capture the new tab, then assert its URL
await page.getByRole('link', { name: 'Size guide' }).click();
```

```
const [guide] = await Promise.all([
  page.context().waitForEvent('page'),
  page.getByRole('link', { name: 'Size guide' }).click(),
]);
await expect(guide).toHaveURL(/size-guide/);
```

Why: the 'page' event hands you the new tab as a Page object. Set the wait up in the same Promise.all as the click so you never miss it.

Exercise 2 – Act Inside a Frame

YOU TRY

The newsletter sign-up is inside an iframe (id 'promo'). A normal locator cannot reach it. Fill the email and submit through the frame.

Your task:

- Scope into the frame with `frameLocator('#promo')`.
- Fill the Email field and click Subscribe inside it.



IN THE IDE go through `frameLocator` before locating the fields.

```
// TODO: scope into '#promo' and interact
const promo = page.frameLocator('#promo');
```

```
const promo = page.frameLocator('#promo');  
await promo.getByLabel('Email').fill('user@test.com');  
await promo.getByRole('button', { name: 'Subscribe' }).click();  
await expect(promo.getByText('Thanks!')).toBeVisible();
```

Why: every locator is taken from the frame handle (promo), so they resolve inside the iframe — the success message is asserted in the frame too.

Exercise 3 — Run an Axe Scan

YOU TRY

Scan the product page for accessibility violations against WCAG 2 A/AA and assert there are none (the seeded page has one to find).

Your task:

- Build an AxeBuilder scan tagged wcag2a + wcag2aa.
- Assert violations is empty; read the one that fails.



IN THE IDE scan, assert empty, then read violations[0] to see what failed.

```
// TODO: new AxeBuilder({ page }).withTags([...]).analyze()  
// then expect(results.violations).toEqual([])
```


24 Exercise 3 — Solution

SOLUTION

```
const results = await new AxeBuilder({ page })
  .withTags(['wcag2a', 'wcag2aa']).analyze();
expect(results.violations).toEqual([]);
// when it fails, the report names the rule, e.g.
// 'image-alt' impact: 'critical'
```

⚠ **Lesson:** a11y failures read like any assertion — rule id + impact + nodes tell you exactly what to fix or file.

PROJECT WORK

Afternoon Lab

Build a real catalog→cart journey with a frame, a dialog, and an accessibility scan — in five staged milestones, each with an acceptance gate.

~4 hours · work in the IDE · trainer circulates

26 Lab Overview

PROJECT WORK

Goal: a navigation spec that goes catalog → product → cart, handles a new tab and an in-frame action, responds to a dialog, and passes an Axe scan on a key page.

1 Navigate the journey

catalog → product → cart, asserting each transition

2 Handle a new tab

Open 'Size guide'; assert the new tab, then close it

3 Act inside a frame

Sign up via the in-page promo iframe

4 Respond to a dialog

Remove a cart item; accept the confirm dialog

5 Accessibility scan

Axe scan the cart page; fix or file one violation

Build the core flow and assert every transition: catalog → product → cart.



IN THE IDE create tests/nav.spec.ts; assert the heading, then the cart count.

```
await page.goto('/catalog');
await page.getByRole('link', { name: 'Running Shoes' }).click();
await expect(page.getByRole('heading', { name: 'Running Shoes' })).toBeVisible();
await page.getByLabel('Size').selectOption({ label: 'UK 9' });
await page.getByRole('button', { name: 'Add to cart' }).click();
await expect(page.getByTestId('cart-count')).toHaveText('1');
```

✓ **Gate:** select a size, add to cart, and assert each transition (heading + cart count).

Open the 'Size guide' in a new tab, assert it, then close it and continue in the original page.



IN THE IDE capture the tab via the 'page' event; assert and close it.

```
const [guide] = await Promise.all([
  page.context().waitForEvent('page'), page.getByRole('link', { name: 'Size guide' }).click(),
]);
await expect(guide).toHaveURL(/size-guide/); await guide.close();
```

✓ **Gate:** the new tab is captured and asserted via the 'page' event; no window indices used.

Sign up through the promo iframe using `frameLocator`, and assert the success message inside the frame.



IN THE IDE scope into '#promo' and interact entirely within the frame.

```
const promo = page.frameLocator('#promo');  
await promo.getByLabel('Email').fill('user@test.com');  
await promo.getByRole('button', { name: 'Subscribe' }).click();  
await expect(promo.getByText('Thanks!')).toBeVisible();
```

✓ **Gate:** the sign-up works through `frameLocator`; the confirmation is asserted inside the frame.

Remove a cart item. A confirm dialog appears — register the handler before the click, accept it, and assert the item is gone.



IN THE IDE `page.once('dialog')` first (assert its text), then click Remove, then assert the empty cart.

```
page.once('dialog', async d => {
  expect(d.message()).toContain('Remove this item?'); await d.accept();
});
await page.getByRole('button', { name: 'Remove' }).click();
await expect(page.getByTestId('cart-count')).toHaveText('0');
```

✓ **Gate:** handler registered with `once()` before the click and asserts the message; cart empties.

31 Milestone 5 – Accessibility Scan

LAB

Run an Axe scan on the cart page. Read any violation, then fix the markup or file it with the rule id.



IN THE IDE scan the cart page tagged wcag2a/aa; assert no violations.

```
const results = await new AxeBuilder({ page })
  .withTags(['wcag2a', 'wcag2aa']).analyze();
expect(results.violations).toEqual([]);
```

✓ **Gate:** the cart page passes the Axe scan, or each violation is named with its rule id and triaged.



Window indices

Do not rely on tab order — capture the 'page' event instead.



Locating into frames

A page locator can't see inside an iframe — use frameLocator.



Late dialog handler

Register page.once('dialog') BEFORE the triggering click.



Assuming navigation

Assert the transition (URL / heading), don't assume it happened.



Custom vs native select

Check the HTML: selectOption only works on real <select>.



a11y = 100% automated

Axe finds many issues, not all — pair it with manual checks.

Before you call the journey done, check it against these.

- ✓ **Every transition asserted?** URL or visible UI checked at each step, not assumed.
- ✓ **Tabs via the 'page' event?** New windows captured as Page objects, no indices.
- ✓ **Frame work scoped?** All in-frame actions go through frameLocator.
- ✓ **Dialog handled in time?** Handler registered before the triggering action.
- ✓ **a11y scanned?** Key page passes Axe, or violations are triaged.

Day 4 contribution areas for the Week 1 mini-capstone. The full rubric is scored out of 100 at the weekly pair demo — here is where each area comes from today:

Concept understanding	Explain context model, frames, dialogs, a11y	15
Hands-on implementation	Working catalog→cart journey in Git	30
Flow & navigation quality	Tabs/frames handled cleanly; transitions asserted	15
Assertions & coverage	Each step + a11y scan asserted	15
Headless / CI readiness	Journey + Axe pass headless	10
Git hygiene	Branch, commit, clean tree	10
Collaboration & demo	Explain the flow & a11y findings	5
Total	Weekly mini-capstone score	100

- ✓ Assert every navigation transition (URL / heading) — don't assume the page changed.
- ✓ New tabs and pop-ups are Page objects in the context — capture the 'page' event, no indices.
- ✓ Reach inside iframes with frameLocator; it chains for nested frames and auto-waits.
- ✓ Drive native `<select>` with `selectOption`; custom menus are click-to-open then click-option.
- ✓ Register `page.once('dialog')` BEFORE the action that triggers the alert / confirm / prompt.
- ✓ Run Axe on key journeys; read rule id + impact — it catches many a11y issues, not all.

36 Quick Reference

RECAP

Pin this — the navigation, frame, dialog and a11y patterns for real-world flows.

Navigation & frames

```
// new tab (set wait before click)
const [tab] = await Promise.all([
  page.context().waitForEvent('page'),
  page.getByRole('link',
    { name: 'Size guide' }).click(),
]);
// act inside a frame
const promo = page.frameLocator('#promo');
await promo.getByRole('button',
  { name: 'Subscribe' }).click();
```

Dialogs & a11y

```
// dialog (register first)
page.once('dialog', async d => {
  expect(d.message())
    .toContain('Remove this item?');
  await d.accept();
});
// native dropdown
await page.getByLabel('Size')
  .selectOption({ label: 'UK 9' });
// accessibility scan
const results = await
  new AxeBuilder({ page }).analyze();
expect(results.violations).toEqual([]);
```

Day 4 complete

You can automate real multi-page flows — tabs, frames, dropdowns and dialogs — and verify accessibility with Axe.



Next — Day 5: Debugging, Stability & Parallel Execution — trace viewer, retries, error handling, and running the suite in parallel.